

Step 1: Design the New Register Map

Goal

Before writing any code, we need to decide what registers our FPGA peripheral exposes and how the 32-bit `writedata` is used.

Concept

A register map is a contract between hardware and software. The FPGA has a small set of "addresses" it responds to. When the ARM writes `iowrite32(value, base + offset)`, the Avalon bus delivers `address` and `writedata` to our peripheral. We get to decide what each address means. With 32-bit `writedata`, we can now pack multiple values into a single write. For example, all three color bytes (24 bits total) fit in one 32-bit register. And a 10-bit X or Y coordinate fits easily.

The New Register Map

Word addr (what address port sees in SV)	Byte offset (what <code>iowrite32</code> uses in C)	Contents	Bits used
0	0x00	Background color: {8'b0, <code>red[7:0]</code> , <code>green[7:0]</code> , <code>blue[7:0]</code> }	[23:0]
1	0x04	Ball X position (0–639)	[15:0]
2	0x08	Ball Y position (0–479)	[15:0]

Why this layout?

- Packing RGB into one register means setting the background is a single `iowrite32()` -- atomic, no half-updated state
- X needs 10 bits (0–639) and Y needs 9 bits (0–479), but we use 16 bits each for clean alignment
- 3 registers with 32-bit words means address port width of 2 bits suffices, but we'll keep 3 bits for future expansion

Transition

Now let's modify the SystemVerilog to implement this register map.

Step 2: Modify `vga_ball.sv` -- The Port Declaration

Goal

Widen `writedata` from 8 to 32 bits so our peripheral can receive full coordinates in a single write.

Concept

The module port list is the hardware interface that Platform Designer wires to the Avalon bus. You already updated Platform Designer to expect 32-bit `writedata`, so now the `.sv` must match.

Code

Change the module header from this:

To this:

Key Point

This is the *only* port change. The VGA output signals, `clk`, `reset`, `chipselct`, `write`, and `address` all stay the same. That's why this was a simple Platform Designer update -- only one signal's width changed.

Step 3: Modify `vga_ball.sv` -- Internal Registers and Latching

Goal

Add `ball_x` and `ball_y` registers and update the `always_ff` block that latches incoming writes.

Concept

The existing code has three 8-bit registers (`background_r`, `background_g`, `background_b`) and assigns them individually from the 8-bit `writedata`. With 32 bits, we can pack all three colors into one write, and we add two new registers for ball position. The `always_ff @(posedge clk)` block fires every clock edge. When `chipselct && write` is true, the Avalon bus is writing to us. The `case (address)` selects *which* register to update.

Code

Replace the current internal registers and `always_ff` block:

Key Lines Explained

- `{background_r, background_g, background_b} <= writedata[23:0]` -- This is a *concatenation assignment*. The 24-bit slice `writedata[23:0]` is split across three 8-bit registers. Bits [23:16] go to `background_r`, [15:8] to `background_g`, [7:0] to `background_b`. One write, three registers updated atomically.
- `ball_x <= writedata[15:0]` -- We only care about the lower 16 bits. The upper 16 bits of `writedata` are ignored. In practice only bits [9:0] matter (max value 639), but taking 16 bits keeps it clean.
- Reset values `320, 240` -- Center of the 640x480 screen, so you see the ball in the middle on boot.

Step 4: Modify `vga_ball.sv` -- Ball Drawing Logic

Goal

Replace the hardcoded ball position with logic that uses our new `ball_x` and `ball_y` registers.

Concept

The VGA counters continuously scan through every pixel. `hcount` runs 0–1599 (at 50 MHz, so two counts per pixel), and `vcount` runs 0–524. The actual visible pixel coordinate is `hcount[10:1]` for X (0–639) and `vcount` for Y (0–479). For each pixel, we ask: "Is this pixel inside the ball?" If yes, draw ball color. If no,

draw background. The current code hardcodes the ball region using bit slicing (`hcount[10:6] == 5'd3`). We'll replace that with an arithmetic comparison against our registers.

Code

Replace the current `always_comb` block:

Key Lines Explained

- `hcount[10:1]` -- This is the actual pixel X coordinate (0–639). Remember `hcount` runs at 2x the pixel rate, so bit 0 is the sub-pixel clock. Dropping it gives us the real column.
- `ball_x[9:0]` -- We declared `ball_x` as 16-bit but only 10 bits are meaningful for a 640-wide screen. Taking `[9:0]` ensures clean 10-bit comparisons with `hcount[10:1]`.
- `>= center - RADIUS` and `< center + RADIUS` -- This defines a square region. For a 16-pixel-diameter ball, `RADIUS` is 8. The ball spans from `(x-8)` to `(x+7)` inclusive.
- `localparam` -- A compile-time constant in SystemVerilog. Changing the ball size later means editing one number.

Caveat

This draws a square, not a circle. That's fine for the lab. If you want a circle later, you'd check `(dx*dx + dy*dy) <= RADIUS*RADIUS`, which adds multipliers but still synthesizes fine at 50 MHz.

Transition

The hardware is done. Now let's update the software stack, starting with the shared header.

Step 5: Modify `vga_ball.h` -- The Shared Data Contract

Goal

Update the data structures and ioctl commands that both kernel and userspace agree on.

Concept

This header is compiled into both the kernel module *and* the userspace program. It defines:

1. The struct layout for data passed through `ioctl()`
1. The `ioctl` command numbers (built with `_IOW/_IOR` macros)

We need to add ball position fields and a new `ioctl` command for setting position.

Code

Replace the entire content of `vga_ball.h`:

Key Lines Explained

- `unsigned short x, y` -- 16-bit unsigned, matches the 16-bit `ball_x/ball_y` in the hardware. Using `unsigned short` (not `int`) keeps the struct small and matches the register width exactly.
- `_IOW(VGA BALL_MAGIC, 3, vga_ball_arg_t)` -- This creates a unique `ioctl` command number. `_IOW` means "write from user to kernel" (the `o` is from the kernel's perspective -- data goes "out" from userspace). The `3` is just the next sequential command number. The magic `'q'` prevents accidental collisions with other drivers' `ioctls`.
- We keep the old commands (1 and 2) -- `WRITE_BACKGROUND` and `READ_BACKGROUND` still work. We're *adding* a new command, not replacing.

Transition

Now the kernel driver needs to handle this new `ioctl` and write to the correct register offsets.

Step 6: Modify `vga_ball.c` -- Register Offset Macros

Goal

Update the macros that compute byte offsets for each FPGA register, and update the device state struct.

Concept

The old driver used `iowrite8()` with byte offsets 0, 1, 2 because `writedata` was 8 bits (1 byte = 1 word). Now with 32-bit words, register N is at byte offset $N * 4$. The driver must use `iowrite32()` and multiply addresses by 4.

Code

Replace the register macros and device struct. Here's the old code:

And the new version:

Key Point

`BG_COLOR(dev.virtbase)` evaluates to `dev.virtbase + 0`. `BALL_X(dev.virtbase)` evaluates to `dev.virtbase + 4`. The +4 is in bytes, and the AXI-to-Avalon bridge divides by the word size (4), so the FPGA's address port sees 1. This is the byte-to-word translation happening transparently.

Step 7: Modify `vga_ball.c` -- Write Functions

Goal

Replace `write_background()` (which used three `iowrite8` calls) with 32-bit write functions.

Concept

`iowrite32(value, address)` writes a 32-bit value to a memory-mapped I/O address. One call replaces three. For the background, we pack R, G, B into bits [23:0] of a 32-bit word -- exactly matching how the hardware's `case` statement unpacks them.

Code

Replace `write_background()` and add a new `write_ball_pos()`:

Key Lines Explained

- `((u32)background->red << 16)` -- Shifts the 8-bit red value to bits [23:16]. The cast to `u32` prevents overflow. The `|` combines all three into one 32-bit value. On the FPGA side, `writedata[23:16]` maps to `background_r`.

- `iowrite32(x, BALL_X(dev.virtbase))` -- Writes the 16-bit X value (zero-extended to 32 bits) to byte offset 4. The FPGA sees `address=1, writedata=x`, and latches `ball_x`.
- `dev.ball_x = x` -- Caches the value in the driver struct so we can read it back via `ioctl` without needing to read from hardware.

Step 8: Modify `vga_ball.c` -- The `ioctl` Handler

Goal

Add a case for the new `VGA BALL SET POS` command.

Concept

The `ioctl` handler is a switch statement on the command number. Userspace calls `ioctl(fd, VGA BALL SET POS, &arg)`, the kernel invokes our handler, we `copy_from_user()` to safely read the struct, then call our write function.

Code

Add the new case inside the existing switch in `vga_ball_ioctl()`:

Key Points

- `copy_from_user()` -- Never trust a userspace pointer directly. This function safely copies the struct from user memory to kernel memory. Returns non-zero on failure (bad pointer, etc.).
- `READ_BACKGROUND` updated -- Now also returns the ball position from the cached values, so userspace can read back the full state.
- New `SET_POS` case -- Same pattern as `WRITE_BACKGROUND`: copy the struct in, call the write function.

Step 9: Update `vga_ball_probe()` Initial Values

Goal

Set a sensible initial ball position when the driver loads.

Code

In `vga_ball_probe()`, after `write_background(&beige)`, add:

This centers the ball on screen when you `insmod vga_ball.ko`.

Summary of All Changes

File	What changed
<code>vga_ball.sv</code>	<code>writedata</code> widened to 32-bit. Added <code>ball_x</code> , <code>ball_y</code> registers. Register latch uses <code>writedata[23:0]</code> for color, <code>writedata[15:0]</code> for position. Ball drawing uses registers instead of hardcoded constants.
<code>vga_ball.h</code>	Added unsigned short <code>x</code> , <code>y</code> to <code>vga_ball_arg_t</code> . Added <code>VGA BALL SET POS</code> ioctl command.
<code>vga_ball.c</code>	Register macros updated to 32-bit byte offsets (0, 4, 8). <code>write_background()</code> uses <code>iowrite32()</code> with packed RGB. New <code>write_ball_pos()</code> . New ioctl case for <code>SET_POS</code> . Probe sets initial ball position.

After making these changes, on the board you'd do:

And you should see a white square ball at the center of the screen against a beige background. The next step would be writing the userspace bouncing program -- want to walk through that next?

Key Lines Explained

`{background_r, background_g, background_b} <= writedata[23:0]` -- This is a concatenation assignment. The 24-bit slice `writedata[23:0]` is split across three 8-bit registers. Bits `[23:16]` go to `background_r`, `[15:8]` to `background_g`, `[7:0]` to `background_b`. One write, three registers updated atomically.

`ball_x <= writedata[15:0]` -- We only care about the lower 16 bits. The upper 16 bits of `writedata` are ignored. In practice only bits `[9:0]` matter (max value 639), but taking 16 bits keeps it clean.

Reset values 320, 240 -- Center of the 640x480 screen, so you see the ball in the middle on boot.

I'm a little bit confused. If background `r` `g` and `b` are also pulling 8 bits each from the `writedata`, how is `x` and `y` supposed to each draw its own bits?

Great question -- this is a really common point of confusion. The key insight is: only one `case` branch fires per write. They never execute at the same time.

Each `iowrite32()` Is a Separate Transaction

Look at the `case` statement:

When software does `iowrite32(color, base + 0)`, the Avalon bus presents:

- `address = 0, writedata = color, chipselect = 1, write = 1`
- Only the `3'h0` branch matches. `ball_x` and `ball_y` are untouched.

When software later does `iowrite32(320, base + 4)`, a *completely separate* bus transaction happens:

- `address = 1, writedata = 320, chipselect = 1, write = 1`
- Only the `3'h1` branch matches. Background R/G/B are untouched.

The Timeline

The three `iowrite32()` calls from the kernel driver happen at different moments in time:

Time 

Clock cycle N: `address=0   writedata=0x00F9E4B7`

→ `background_r=F9, background_g=E4, background_b=B7`

→ `ball_x` unchanged, `ball_y` unchanged

Clock cycle N+?: `address=1   writedata=0x00000140   (320 decimal)`

→ `ball_x=320`

→ background unchanged, `ball_y` unchanged

Clock cycle N+?: `address=2   writedata=0x000000F0   (240 decimal)`

→ `ball_y=240`

→ background unchanged, `ball_x` unchanged

Each register is independently addressed. The 32-bit `writedata` bus carries different data each time -- it's the `address` that determines *which* register receives it.

Analogy

Think of it like a mailroom with three mailboxes (0, 1, 2). Each delivery comes with an address label and a package. The mailroom clerk looks at the label and puts the package in that one box. The other boxes don't get touched. The same delivery slot is used every time, but the address label changes.